Linked List – Introduction

- A **Linked List** is a linear data structure consisting of a collection of nodes.
- Each node contains:
  - **Data:** Stores the actual value.
  - **Reference (Link):** Stores the reference to the next node (and previous node in a doubly linked list).
- Nodes are dynamically allocated and are not required to be stored in contiguous memory locations.
- The first node is accessed using a reference called **head**.
- The last node of a normal singly linked list points to `null`.
- Linked lists support dynamic memory allocation, allowing the size to grow or shrink during execution.
- Unlike arrays, linked lists do not support direct/random access to elements.

Types of Linked List

1. Singly Linked List (SLL)

- Each node contains `data` and `next`.
- `next` stores the reference to the next node.
- The last node points to `null`.
- Traversal is possible only in the forward direction.
- Requires less memory compared to a doubly linked list.

2. Doubly Linked List (DLL)

- Each node contains `prev`, `data`, and `next`.
- `prev` stores the reference to the previous node.
- `next` stores the reference to the next node.
- Supports traversal in both forward and backward directions.
- Requires extra memory for storing the `prev` reference.

3. Circular Singly Linked List (CSLL)

- Similar to a singly linked list, but the last node points back to the first node (`head`).

SUNBEAM

- The last node does not contain `null` in its `next` reference.
- Traversal can start from any node and continue circularly.
- Useful for applications requiring repeated or cyclic traversal.

4. Circular Doubly Linked List (CDLL)

- Each node contains `prev`, `data`, and `next`.
- The last node's `next` points to the first node.
- The first node's `prev` points to the last node.
- Supports circular traversal in both forward and backward directions.
- Requires more memory because each node maintains two references.

Advantages of Linked List

- **Dynamic Size:** Size can grow or shrink during program execution.
- **Efficient Insertion and Deletion:** No need to shift elements as in arrays.
- **Memory Utilization:** Memory is allocated dynamically as required.
- **Non-contiguous Memory:** Nodes can be stored at different memory locations.
- **Easy Implementation of Other Data Structures:** Can be used to implement stacks, queues, etc.
- **Flexible Structure:** Nodes can be easily rearranged by changing references.

Disadvantages of Linked List

- **No Random Access:** Direct access to an element using an index is not possible.
- **Sequential Access:** Elements must be traversed one by one.
- **Extra Memory:** Additional memory is required to store references (`next` and `prev`).
- **Poor Cache Locality:** Nodes may be scattered in memory, resulting in less efficient cache utilization compared to arrays.
- **Reverse Traversal:** Not directly possible in a singly linked list.
- **Complex Pointer Management:** Incorrect reference updates may cause loss of nodes or incorrect links.
- **Searching:** Searching an element in an unsorted linked list takes O(n) time.

Applications of Linked List

SUNBEAM

- **Stack Implementation:** Used to implement dynamic stacks.
- **Queue Implementation:** Used to implement dynamic queues.
- **Hash Tables:** Separate chaining uses linked lists to handle collisions.
- **Graph Representation:** Adjacency lists use linked lists to store neighbouring vertices.
- **Polynomial Representation:** Each node can represent a polynomial term (coefficient and exponent).
- **Sparse Matrix Representation:** Stores only non-zero elements to save memory.
- **Undo/Redo Operations:** Doubly linked lists can maintain previous and next states.
- **Browser Navigation:** Doubly linked lists can support forward and backward navigation.
- **Round Robin Scheduling:** Circular linked lists can be used to manage processes cyclically.
- **Music Playlist:** Circular or doubly linked lists can support continuous playback and navigation.

Floyd's Cycle Detection Algorithm (Tortoise and Hare Algorithm)

1. Introduction

- Floyd's Cycle Detection Algorithm is used to detect whether a **cycle (loop) exists in a linked list**.
- It uses two pointers:
  - **Slow Pointer (Tortoise):** Moves one node at a time.
  - **Fast Pointer (Hare):** Moves two nodes at a time.
- If a cycle exists, both pointers will eventually meet at the same node.
- If the fast pointer reaches `null`, the linked list does not contain a cycle.
- It does not require extra memory for storing visited nodes.

2. Working

- Initialize both `slow` and `fast` pointers to `head`.
- Move `slow` by one node in each iteration.
- Move `fast` by two nodes in each iteration.
- Compare both pointers after moving.
- If `slow == fast`, a cycle is detected.
- If `fast == null` or `fast.next == null`, no cycle exists.

SUNBEAM

3. Important Java Conditions

- Pointer initialization:
  - `slow = head`
  - `fast = head`
- Loop condition:
  - `fast != null && fast.next != null`
- Pointer movement:
  - `slow = slow.next`
  - `fast = fast.next.next`
- Cycle detection condition:
  - `slow == fast`
- **Important:** Move both pointers before checking whether they meet. Otherwise, when both start at `head`, a false cycle detection may occur.

4. Time and Space Complexity

- **Time Complexity:** O(n)
  - Each pointer traverses the linked list at a bounded rate.
- **Space Complexity:** O(1)
  - Only two additional references are used.
- It is more space-efficient than using a HashSet to store visited nodes.

5. Advantages

- Detects cycles in a linked list efficiently.
- Requires constant extra space: O(1).
- Does not modify the original linked list.

SUNBEAM

- Simple and efficient compared to storing visited nodes.

6. Disadvantages

- Mainly applicable to detecting cycles in linked structures.
- Does not directly provide the complete list of nodes forming the cycle.
- Additional steps are required to find the cycle's starting node and its length.

7. Applications

- Detecting loops in singly linked lists.
- Finding the starting node of a cycle.
- Finding the length of a cycle.
- Detecting infinite loops in linked data structures.
- Used in algorithmic problems involving cyclic sequences.

8. Important MCQ Points

- Floyd's algorithm is also called **Tortoise and Hare Algorithm**.
- Slow pointer moves **1 step** per iteration.
- Fast pointer moves **2 steps** per iteration.
- If both pointers meet, a cycle exists.
- If `fast == null` or `fast.next == null`, no cycle exists.
- Time complexity: **O(n)**.
- Auxiliary space complexity: **O(1)**.
- The algorithm does not require a HashSet or visited array.
- Both pointers are initially set to `head`.
- A cycle can exist even if the linked list does not contain a `null` reference.
- After detecting a cycle, resetting one pointer to `head` and moving both pointers one step at a time helps find the **starting node of the cycle**.
- To calculate cycle length:
  - Keep one pointer fixed at the meeting node.
  - Move another pointer one step at a time until it returns to the meeting node.
  - Count the number of steps.

SUNBEAM